

■ nezuko Linux+ Study Notes

Section 9 — Users, Groups & Permissions

Server: **nezuko** | User: **scottp** | OS: Ubuntu 26.04 LTS (Resolute)

Certification Track: **CompTIA Linux+**

■ Overview & Learning Goals

Every file on a Linux system has an **owner** and a set of **permissions** that control exactly who can read it, write to it, or execute it. Understanding this is not just an exam topic — it is the single most common cause of real-world 'permission denied' errors you will encounter as a DevOps/DevSecOps engineer.

By the end of this section you will be able to:

- Identify the current user and all group memberships
- Create, inspect and delete users and groups
- Read and decode ls -l permission strings
- Change file permissions using both symbolic and numeric (octal) mode
- Change file ownership with chown and chgrp
- Use sudo safely and understand root vs normal users
- Explain the /etc/passwd, /etc/shadow, and /etc/group file formats

7.1 Users & Groups

■ whoami — Display the currently logged-in username

Prints the username of the account running the current shell session. Simple and instant — useful to confirm who you are, especially after switching users with `sudo -u`.

Common Options

Option	
(none)	No flags needed — just run <code>whoami</code>

Examples on nezuko

```
scottp@nezuko:~$ whoami
scottp                      # Shows your normal user account

scottp@nezuko:~$ sudo -i
root@nezuko:~# whoami
root                       # Confirms you are now root after sudo -i
```

■ LINUX+ EXAM TIP

`whoami` is often tested in questions about identifying the current user. Root always returns 'root'. UID 0 = root.

■ id — Display user identity: UID, GID, and all group memberships

Shows the numeric and named UID, primary GID, and every supplementary group for a user. More detailed than `whoami` — essential for diagnosing permission issues because Linux uses the numeric IDs internally, not names.

Common Options

Option	
(none)	Show current user's full identity
username	Show identity for a specific user (e.g. <code>id scottp</code>)
-u	Print only the UID number
-g	Print only the primary GID number
-G	Print all group IDs (space-separated)
-n	Print names instead of numbers (use with -u, -g, -G)

Examples on nezuko

```
scottp@nezuko:~$ id
uid=1000(scottp) gid=1000(scottp) groups=1000(scottp),4(adm),27(sudo),
    998(docker)    # Shows ALL groups – confirms docker and sudo membership

scottp@nezuko:~$ id -u
1000              # Numeric UID only

scottp@nezuko:~$ id -Gn
scottp adm sudo docker # All group names

scottp@nezuko:~$ id root
uid=0(root) gid=0(root) groups=0(root) # Root always UID 0
```

■ LINUX+ EXAM TIP

Linux+ loves UID/GID questions. Root = UID 0. Normal users start at UID 1000 on most modern distros. System accounts use UIDs below 1000.

■ groups — List all groups the current user belongs to

Prints group names (not numbers) for the current user or a named user. Simpler than `id` when you only need group names.

Common Options

Usage	
<code>groups</code>	List groups for the current user
<code>groups username</code>	List groups for a specific user (e.g. <code>groups scottp</code>)

Examples on nezuko

```
scottp@nezuko:~$ groups
scottp adm sudo docker    # scottp belongs to these groups

scottp@nezuko:~$ groups root
root                      # root only belongs to its own group
```

■ LINUX+ EXAM TIP

Being in the 'sudo' group grants sudo access. Being in 'docker' group lets you run Docker without sudo — a common privilege-escalation vector.

■ adduser — Interactively create a new user account

The friendly, high-level user-creation command on Debian/Ubuntu systems. Automatically creates the home directory, sets a password interactively, copies skeleton files from /etc/skel, and creates a matching group. Contrast with `useradd` which is the lower-level command (more control, no interactivity by default).

Common Options

Option	
<code>adduser username</code>	Create a new normal user (interactive prompts)
<code>adduser user group</code>	Add an existing user to an existing group
<code>--disabled-password</code>	Create account without a password (useful for service accounts)
<code>--home /path</code>	Set a custom home directory path
<code>--shell /bin/bash</code>	Set a specific login shell
<code>--gecos 'Full Name'</code>	Set the comment/full name field non-interactively

Examples on nezuko

```
scottp@nezuko:~$ sudo adduser testuser
# Creates testuser with home /home/testuser, prompts for password

scottp@nezuko:~$ sudo adduser testuser sudo
# Adds testuser to the sudo group (grants sudo privileges)

scottp@nezuko:~$ cat /etc/passwd | grep testuser
testuser:x:1001:1001:./home/testuser:/bin/bash
# Verify the new account was created
```

■ LINUX+ EXAM TIP

Linux+ distinguishes `adduser` (Debian/Ubuntu, interactive, high-level) from `useradd` (universal, low-level, non-interactive). Know both exist. ac

■ deluser — Remove a user account

Removes a user account. By default does NOT delete the home directory — use `--remove-home` to also purge their files. This is intentional: you may need to recover files before deleting them.

Common Options

Option	
<code>deluser username</code>	Remove the user (keeps home directory)
<code>--remove-home</code>	Also deletes <code>/home/username</code> and mail spool
<code>--backup</code>	Back up files to current directory before deleting
<code>--remove-all-files</code>	Delete ALL files on system owned by this user (dangerous)
<code>deluser user group</code>	Remove user from a group (does not delete the account)

Examples on nezuko

```
scott@nezuko:~$ sudo deluser testuser
# Removes account but /home/testuser still exists

scott@nezuko:~$ sudo deluser --remove-home testuser
# Removes account AND deletes /home/testuser completely

scott@nezuko:~$ sudo deluser scott sudo
# Removes scott from the sudo group only – does NOT delete scott
```

■ WARNING

Never delete a user that owns running processes or services.
Always check: `ps aux | grep username` before deleting.
Use `--backup` if there is any chance you need their files later.

Key System Files — User & Group Databases

■ /etc/passwd — User account database

The primary user database. Every account on the system — including service accounts like www-data and nobody — has a line here. It is world-readable because many programs need to look up user information. Passwords are NOT stored here (they live in /etc/shadow).

Permissions

Who		
root	rw	Full control
group	r	System group reads for lookups
others	r	World-readable — needed by many programs

Line format (7 colon-separated fields)

```
username : x : uid : gid : comment : home_dir : shell
scottp   : x : 1000: 1000: Scott P : /home/scottp : /bin/bash
```

Field Breakdown

Field		
username	scottp	Login name
password	x	'x' means password is in /etc/shadow
uid	1000	Unique user ID number
gid	1000	Primary group ID
comment	Scott P	Full name / GECOS field (informational)
home	/home/scottp	Home directory path
shell	/bin/bash	Default login shell

■ LINUX+ EXAM TIP

The 'x' in the password field is critical — it means the actual (hashed) password is stored in /etc/shadow. If this field were empty, the account

■ **/etc/shadow — Secure password storage**

Contains hashed passwords and password-ageing information. Readable only by root (and the shadow group on some systems). This separation from /etc/passwd prevents non-root users from obtaining password hashes to crack offline.

Permissions

Who		
root	rw	Must be able to update passwords
shadow grp	r	Some tools like su read via this group
others	none	No access — protects password hashes from brute-force

Example entry

```
scottp:$6$rounds=...$hashedpassword:19500:0:99999:7:::
#           ^hash algo^                ^last change^ ^max age^
```

■ **LINUX+ EXAM TIP**

\$6\$ = SHA-512 hash algorithm. \$5\$ = SHA-256. \$1\$ = MD5 (old, insecure). The Linux+ exam may ask you to identify hash algorithms from /

■ /etc/group — Group database

Lists every group on the system, its GID, and its members. A user's primary group is set in /etc/passwd; supplementary (secondary) groups are listed here.

Line format (4 colon-separated fields)

```
groupname : x : gid : member1,member2,...
sudo      : x : 27  : scottp
docker    : x : 998 : scottp
```

Field Breakdown

Field		
groupname	sudo	Group name
password	x	Rarely used — placeholder
gid	27	Numeric group ID
members	scottp	Comma-separated list of member usernames

■ LINUX+ EXAM TIP

A user can belong to ONE primary group (from /etc/passwd) and many supplementary groups (from /etc/group). Group membership change

7.2 File Permissions

■ Concept — Reading ls -l Permission Strings

When you run `ls -l`, the first column is a 10-character string that encodes everything about a file's type and permissions. Think of it as a license plate for the file.

Position(s)		
1	-	File type: - = regular file, d = directory, l = symbolic link, c = char device, b = block device
2-4	rw-	Owner permissions: r=read w=write x=execute -=no permission
5-7	r--	Group permissions
8-10	r--	Others (world) permissions
11 (if +)	+	ACL or SELinux attribute is set (beyond basic permissions)

Full example decoded

```
-rw-r--r-- 1 root root 1842 Dec 10 14:30 /etc/passwd
#  ^^  ^^^  ^^^
#  |  |  |--- others: r-- = read only
#  |  |----- group:  r-- = read only
#  |----- owner: rw- = read + write (no execute)
#
drwxr-xr-x 2 scottp scottp 4096 Jan 01 09:00 /home/scottp
#  ^^^ start with 'd' = DIRECTORY
#  rwx = owner can read, write, AND enter (cd into) the directory
#  r-x = group and others can list and enter, but cannot create/delete files inside
```

What r / w / x actually mean (files vs directories)

Permission		
r (read)	View file contents (cat, less)	List contents (ls)
w (write)	Modify / overwrite file contents	Create, delete, rename files inside
x (execute)	Run as a program / script	Enter the directory (cd)
- (none)	Permission denied for that action	Permission denied for that action

■ LINUX+ EXAM TIP

x on a directory means 'traverse' — you need it to cd into a directory. You can have r (list) without x (enter), but listing gives filenames only, n

chmod — Change File Permissions

chmod (change mode) modifies the permission bits of a file or directory. You must own the file (or be root) to change its permissions. Two modes exist: **symbolic** (human-readable) and **numeric/octal** (concise).

Symbolic Mode

Symbolic mode uses letters to specify who is affected and what change to make. Format: chmod [who][operator][permissions] file

Who / Operator / Permission symbols

Symbol		
u	Who	User (owner)
g	Who	Group
o	Who	Others (world)
a	Who	All (u + g + o combined)
+	Operator	Add the permission
-	Operator	Remove the permission
=	Operator	Set EXACTLY these permissions (removes others)
r	Permission	Read
w	Permission	Write
x	Permission	Execute

Symbolic Examples on nezuko

```
chmod u+x deploy.sh      # Add execute for owner – make a script runnable
chmod g+w logs/           # Allow group to write inside logs directory
chmod o-r private.txt    # Remove read from others – tighten security
chmod a+r README.md      # Everyone can read the file (add to all)
chmod u+rw,g-rwx key     # Owner full access; group/others: no access at all
chmod a=r config.txt     # Set EXACTLY read for everyone (no write/execute)
chmod u=rw,g=r,o=        # Owner rw, group r, others nothing
```

Numeric (Octal) Mode

Each permission has a numeric value: r=4, w=2, x=1. Add them up to get the octal digit for each permission group (owner / group / others). Three digits cover all three groups. This is faster and less ambiguous than symbolic mode.

Permission value table

Octal Digit			
7	111	rwX	Read + Write + Execute
6	110	rw-	Read + Write
5	101	r-x	Read + Execute
4	100	r--	Read only
3	011	-wX	Write + Execute
2	010	-w-	Write only
1	001	--X	Execute only
0	000	---	No permissions

Common chmod combinations — memorise these for the exam

Command		
chmod 755 file	rwXr-xr-x	Executable scripts / programs — owner full, rest can read+run
chmod 644 file	rw-r--r--	Regular files — owner rw, rest read-only (web files, configs)
chmod 600 file	rw-----	Private files — SSH keys, .env files, secrets
chmod 700 dir	rwX-----	Private directory — only owner can enter
chmod 777 file	rwXrwXrwX	Full access for everyone — avoid unless absolutely necessary
chmod 664 file	rw-rw-r--	Collaborative file — owner and group can write
chmod 400 file	r-----	Read-only for owner — protect important files from accidents

Numeric Examples on nezuko

```

chmod 755 /home/scottt/scripts/backup.sh
# Owner: rwX Group: r-x Others: r-x - standard executable script

chmod 600 /home/scottt/.ssh/id_rsa
# Owner: rw- Group: --- Others: --- - SSH private key must be 600 or SSH refuses it

chmod 644 /var/www/html/index.html
# Owner: rw- Group: r-- Others: r-- - web file readable by www-data

chmod -R 755 /home/scottt/scripts/
# -R = recursive - apply 755 to directory AND all files inside

```

Option	
-R	Recursive — apply permissions to directory and all contents
-v	Verbose — print each file as it is changed
--reference=file	Copy permissions from another file

■ LINUX+ EXAM TIP

The Linux+ exam heavily tests octal notation. Practise converting: 754 = rwXr-xr-- | 640 = rw-r----- | 421 = r---w---x. Binary helps: each perm

■ chown — Change file owner (and optionally group)

Changes which user owns a file. Only root can change ownership (you cannot give away your own files to another user without sudo). Ownership controls which permission set (owner/group/others) applies to each user.

Common Options

Option/Syntax	
chown user file	Change owner to 'user'
chown user:group file	Change owner AND group
chown :group file	Change group only (same as chgrp)
-R	Recursive — apply to directory and all contents
-v	Verbose output — show each change
--from=owner:group	Only change if current owner:group matches

Examples on nezuko

```
sudo chown scottp /tmp/file.txt
# Transfer ownership to scottp

sudo chown scottp:docker /home/scottp/docker-data/
# Owner = scottp, Group = docker (useful for Docker volume access)

sudo chown -R www-data:www-data /var/www/html/
# Recursively set web server ownership on all web files

sudo chown -R scottp:scottp /home/scottp/
# Fix ownership after a root-run process created files in your home dir
```

■ LINUX+ EXAM TIP

chown user:group (colon) is the standard. Some older docs use chown user.group (dot) — both work but colon is preferred. Only root can ch

■ chgrp — Change the group ownership of a file

Changes only the group owner of a file. You can use `chown :group` to do the same thing — `chgrp` is the dedicated command for group-only changes.

Common Options

Option	
<code>chgrp group file</code>	Change group of file
<code>-R</code>	Recursive
<code>-v</code>	Verbose

Examples on nezuko

```
sudo chgrp docker /var/run/docker.sock
# Allow docker group to access the Docker socket

sudo chgrp -R scottp /media/movies/
# Recursively change group on entire Jellyfin media directory
```

7.3 Sudo & Root

■ Concept — The Root User

Root (also called the superuser) is the all-powerful account on every Linux system. It has UID 0 and bypasses all permission checks — it can read any file, write to any file, kill any process, and change any setting. Think of it as a master key that opens every lock, ignores every guard.

Why not just log in as root all the time? Because one typo can destroy the entire system with no warning. If malware infects a root session, it owns everything. Best practice: use `sudo` for individual commands that need elevation.

■ WARNING

Never run everyday tasks as root. Never leave a root shell open unattended.
If you see prompt `'#'` instead of `'$'` — you are root. Be careful.
`sudo -i` and `sudo su` both give you a root shell. Exit immediately when done.

■ sudo — Execute a command with elevated (root) privileges

sudo (superuser do) lets authorised users run a single command as root (or as another user). You enter YOUR OWN password, not the root password. After authentication, sudo remembers for ~15 minutes (configurable).

Common Options

Option	
sudo command	Run command as root
sudo -u user cmd	Run command as a specific user (not root)
sudo -i	Open a root login shell (loads root's environment)
sudo -s	Open a root shell (keeps current environment)
sudo -l	List what sudo commands you are allowed to run
sudo -k	Forget the cached password immediately
sudo -v	Extend the sudo timeout without running a command
sudo !!	Re-run the last command with sudo (bash only)

Examples on nezuko

```
sudo apt update && sudo apt upgrade -y
# Run package updates as root

sudo -u testuser cat /tmp/secret.txt
# Run cat AS testuser – tests what testuser can access

sudo cat /etc/shadow
# Read the shadow password file (root only)

sudo -i
# Open a root shell – prompt changes from $ to #
# Type 'exit' to return to scottp

sudo -l
# Show all sudo permissions for scottp on nezuko

sudo -k
# Force re-authentication on the next sudo command
```

■ LINUX+ EXAM TIP

sudo -u user runs as a specific user — used in the transcript to prove permissions work. sudo -i gives a full root login shell (sources /etc/profile).

■ /etc/sudoers — Sudo access configuration

Defines exactly who can use sudo, which commands they can run, and whether they need to enter a password.

Never edit this file directly with a text editor — always use visudo which validates the syntax before saving (a syntax error here locks everyone out of sudo).

Key lines in /etc/sudoers

```
root    ALL=(ALL:ALL) ALL
# root can run any command as any user on any host

%sudo   ALL=(ALL:ALL) ALL
# % means a group – the sudo group can run any command

scottp  ALL=(ALL) NOPASSWD: /usr/bin/apt
# scottp can run apt without a password (useful on nezuko)
```

■ WARNING

Always use: `sudo visudo` — NEVER: `sudo nano /etc/sudoers`
A typo in /etc/sudoers can completely lock out sudo on the system.
visudo checks syntax before saving and prevents this.

■ LINUX+ EXAM TIP

Linux+ expects you to know that visudo is the correct tool for editing sudoers. The %groupname syntax in sudoers means 'all members of this group'.

■ Additional Commands — User & Group Management

■ ADDITIONAL CONTENT

The transcript covers adduser and deluser (Debian/Ubuntu friendly wrappers).

The Linux+ exam also tests useradd, usermod, userdel, groupadd, groupdel, and groupmod because these are the universal POSIX-compliant commands found on ALL Linux distributions. Knowing both sets is essential for the certification.

■ useradd — Low-level user creation (universal, non-interactive)

The lower-level, universal user creation command. Unlike adduser, it does NOT prompt for a password or create a home directory by default — you must specify these explicitly with flags. Used in scripts and non-interactive environments.

Common Options

Option	
-m	Create home directory (required — not done by default)
-d /path	Set home directory path explicitly
-s /bin/bash	Set login shell
-g groupname	Set primary group
-G grp1,grp2	Set supplementary groups
-u uid	Set a specific UID
-c 'Full Name'	Set the comment/GECOS field
-r	Create a system account (UID < 1000, no home dir)
-e YYYY-MM-DD	Set account expiry date

Examples on nezuko

```
sudo useradd -m -s /bin/bash -G sudo,docker deployuser
# Create deployuser with home dir, bash shell, in sudo and docker groups

sudo passwd deployuser
# Set password (useradd does not prompt – must do separately)

sudo useradd -r -s /usr/sbin/nologin jellyfin
# Create system service account – no login shell, no home dir
```

■ LINUX+ EXAM TIP

useradd vs adduser: useradd is universal (works on RHEL, Arch, Debian). adduser is a Debian/Ubuntu wrapper around useradd. useradd -m

■ usermod — Modify an existing user account

Modifies properties of an existing account. This is how you add a user to a group, change their shell, lock their account, or rename them — without deleting and recreating.

Common Options

Option	
-aG group	Append user to supplementary group (CRITICAL: use -a or you overwrite all groups)
-g group	Change primary group
-s /bin/bash	Change login shell
-d /path	Change home directory
-m	Move home directory contents (use with -d)
-L	Lock account (prepends ! to password in shadow)
-U	Unlock account
-e YYYY-MM-DD	Set account expiry date
-l newname	Rename the user login name

Examples on nezuko

```
sudo usermod -aG docker scottcp
# Add scottcp to docker group – MUST use -a (append) or all groups are replaced

sudo usermod -aG sudo newuser
# Grant sudo access to newuser

sudo usermod -L baduser
# Lock baduser's account immediately (they cannot log in)

sudo usermod -s /usr/sbin/nologin serviceacct
# Prevent a service account from getting a login shell
```

■ WARNING

usermod -G group (without -a) REPLACES all supplementary groups.
Always use: `sudo usermod -aG groupname username`
Forgetting -a is one of the most common Linux admin mistakes.

■ groupadd / groupdel / groupmod — Manage groups

Create, delete, and modify groups directly. Equivalent to adduser/deluser but operating on groups.

Common Commands & Options

Command	
groupadd groupname	Create a new group
groupadd -g 1500 groupname	Create group with specific GID
groupadd -r groupname	Create a system group (GID < 1000)
groupdel groupname	Delete a group (does not delete members)
groupmod -n newname old	Rename a group
groupmod -g 1600 group	Change a group's GID

Examples on nezuko

```
sudo groupadd media
# Create a 'media' group for Jellyfin media file access

sudo usermod -aG media scottp
# Add scottp to the media group

sudo groupdel media
# Delete the media group (scottp remains – their account is unaffected)
```

■ ADDITIONAL CONTENT

The transcript covers standard rwx permissions. Linux+ also tests three special permission bits: SUID, SGID, and the Sticky Bit. These appear on the exam and are directly relevant to understanding why commands like passwd can change /etc/shadow even though it is owned by root.

■ Concept — Special Permission Bits: SUID, SGID, Sticky Bit

SetUID (SUID) — When set on an executable, it runs with the file OWNER's privileges, not the caller's. Example: `/usr/bin/passwd` is owned by root with SUID set — this is why a normal user can change their own password (which requires writing to root-owned `/etc/shadow`).

SetGID (SGID) — On an executable: runs with the file's GROUP privileges. On a directory: new files created inside inherit the directory's group (very useful for shared project directories).

Sticky Bit — On a directory, only the file's owner (or root) can delete or rename files inside, even if others have write permission. Classic example: `/tmp` — everyone can write there but you cannot delete others' files.

How they appear in `ls -l`

Bit			
SUID	rws (owner x→s)	chmod 4755	<code>/usr/bin/passwd</code>
SGID	rws (group x→s)	chmod 2755	<code>/usr/bin/wall</code>
Sticky	rwt (others x→t)	chmod 1777	<code>/tmp</code>
SUID (no x)	rwS (uppercase S)	chmod 4644	Rare — x not set

Examples on nezuko

```
ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 68208 ... /usr/bin/passwd
# The 's' in owner execute position = SUID set

ls -ld /tmp
drwxrwxrwt 10 root root 4096 ... /tmp
# The 't' in others execute position = Sticky bit

chmod 4755 myscript.sh    # Set SUID (4) + 755
chmod 2775 shared_dir/    # Set SGID (2) + 775 – shared project dir
chmod 1777 shared_tmp/    # Sticky (1) + 777 – world-write but owner-delete only

find / -perm -4000 2>/dev/null # Find all SUID files on system
```

■ LINUX+ EXAM TIP

SUID on `/usr/bin/passwd` is a classic Linux+ question. SUID = 4, SGID = 2, Sticky = 1. Uppercase S or T in `ls -l` means the bit is set but execute

Permissions in Action — Practical Walkthrough

This mirrors the challenge from the transcript, annotated with explanations. Run this on nezuko to prove you understand permissions end to end.

```
# 1. Create test user
sudo adduser alice
# Creates /home/alice, prompts for password

# 2. Create a private file in /tmp (world-accessible directory)
echo 'Secret data' > /tmp/private.txt
chmod 600 /tmp/private.txt
ls -l /tmp/private.txt
# Expected: -rw----- 1 scottp scottp ... /tmp/private.txt

# 3. Prove alice CANNOT read it
sudo -u alice cat /tmp/private.txt
# Expected: cat: /tmp/private.txt: Permission denied

# 4. Make it readable by others
chmod 644 /tmp/private.txt

# 5. Prove alice CAN now read it
sudo -u alice cat /tmp/private.txt
# Expected: Secret data

# 6. Clean up
rm /tmp/private.txt
sudo deluser --remove-home alice
```

Why /tmp? Your home directory (/home/scottp) has permissions 700 by default — alice cannot even enter it to reach a file. /tmp is 1777 (sticky bit + world-write) so alice can reach the file and we test only the file's own permissions.

Linux+ Exam Tips Summary — Section 9

■ UIDs & GIDs

Root = UID 0 always. Normal users start at UID 1000 on modern distros. System/service accounts use UIDs < 1000. `id` command shows all

■ Password storage

`/etc/passwd` is world-readable but stores only 'x' — passwords hash in `/etc/shadow` (root-readable only). `$6$` = SHA-512, `$5$` = SHA-256, `$1$`

■ adduser vs useradd

`adduser` = Debian/Ubuntu interactive wrapper. `useradd` = universal, low-level. `useradd -m` required for home directory. Both appear on the ex

■ chmod octal — memorise

755 = `rxr-xr-x` (executables), 644 = `rw-r--r--` (regular files), 600 = `rw-----` (private/SSH keys), 700 = `rw-x-----` (private dirs), 777 = `rxwxrwxrwx`

■ chmod symbolic

`u/g/o/a + +/=/- + r/w/x`. Example: `chmod u+x,go-w file`. 'a' applies to all three groups at once.

■ Execute on directories

x on a directory = traverse/enter (`cd`). r = list filenames. You need BOTH r AND x to list with metadata. Common exam trap.

■ chown syntax

`chown user:group file` — colon separator. Only root can change ownership. `chown :group` = change group only (same as `chgrp`).

■ usermod -aG

ALWAYS use -a (append) when adding a user to a group. Without -a, all existing supplementary groups are replaced. Classic exam trick que

■ sudoers

Edit with `visudo` ONLY — validates syntax before saving. A syntax error in `/etc/sudoers` can lock all sudo access.

■ Special bits

SUID=4 (s in owner x), SGID=2 (s in group x), Sticky=1 (t in others x). Uppercase S/T = bit set but x not set. `/usr/bin/passwd` has SUID. `/tmp`

■ sudo -u

Runs a command as a specific user — key for testing permissions: `sudo -u alice cat file`

■ visudo & /etc/sudoers

`%groupname` in `sudoers` means all members of that group. NOPASSWD: lets a user run commands without entering a password.

Quick Reference — Key Terms

Term	
UID	User ID — unique number for each user. Root = 0.
GID	Group ID — unique number for each group.
Owner	The user who owns a file and controls its permissions.
Group	A named set of users that share access to files.
Permission	r/w/x bits controlling read, write, execute access.
SUID	SetUID — executable runs with owner's privileges.
SGID	SetGID — executable or directory with group inheritance.
Sticky bit	Directory flag: only owner can delete their own files.
sudo	Superuser do — run one command with elevated privileges.
chmod	Change mode — modify permission bits on a file.
chown	Change owner — modify user/group ownership of a file.
chgrp	Change group — modify group ownership only.
visudo	Safe editor for /etc/sudoers — validates syntax on save.
adduser	Debian/Ubuntu interactive user creation wrapper.
useradd	Universal low-level user creation command.
usermod	Modify existing user account properties.
deluser	Debian/Ubuntu user deletion wrapper.
/etc/passwd	World-readable user account database (no passwords).
/etc/shadow	Root-only file containing hashed passwords.
/etc/group	Group database — lists all groups and their members.

End of Section 9 — Users, Groups & Permissions | nezuko Linux+ Study Notes